12
Grids

> It is not that polar coordinates are complicated, it is just that Cartesian coordinates are simpler than they have a right to be. – Kleppner and Kolenhow, "An Introduction to Mechanics"

Grids underlie a wide variety of natural structures. Chessboards are grids. City blocks are typically arranged on a grid; indeed, the most natural grid distance measure grid is often called the "Manhattan" distance. The system of longitude and latitude defines a grid over the earth, albeit on the surface of a sphere instead of the plane.

Grids are ubiquitous because they are the most natural way to carve space up into regions so that locations can be identified. In the limit, these cells can be individual points, but here we deal with coarser grids whose cells are big enough to have a shape. In *regular* grids, each of these shapes is identical, and they occur in a regular pattern. Rectangular or *rectilinear* subdivisions are the most common grids, due to their simplicity, but triangle-based *hexagonal* grids are also important. Indeed, the honey industry has exploited the efficiency of hexagonal grids for literally millions of years.

12.1 Rectilinear Grids

Rectilinear grids are familiar to anyone who has used a piece of graph paper. In such grids, the cells are typically defined by regularly spaced horizontal and vertical lines. Non-uniform spacing still yields a regular topology, although the size of each cell may differ. Three-dimensional grids are formed by connected regularly spaced layers of planar grids with perpendicular lines across the layers. Three-dimensional grids also have planar faces, defined between any two face-neighboring cubes.

There are three important components of the planar grid: the vertices, the edges, and the cell interiors. Sometimes we are interested in the interiors of the cells, as in geometric applications where each cell describes a region in space. Sometimes we are interested in the vertices of the grid, such as in addressing the pieces on a chessboard. Sometimes we are interested in the edges of the grid, such as when finding routes to travel in a city where buildings occupy the interior of the cells.

Vertices in planar grids each touch four edges and the interiors of four cells, except for vertices on the boundaries. Vertices in 3D grids touch on six edges and eight cells. In d-dimensions, each vertex touches $2d$ edges and 2^d cells. Cells in a planar grid each touch eight faces, four diagonally through vertices and four through edges. Cells in a 3D grid each touch 26 other cells, sharing a face with 6 of them, an edge with 12 of them, and just a vertex with the other 8.

12.1.1 Traversal

It is often necessary to traverse all the cells of an $n \times m$ rectilinear grid. Any such traversal can be thought of as a mapping from each of the nm ordered pairs to a unique integer from 1 to nm. In certain applications the order matters, such as in dynamic programming evaluation strategies. The most important traversal methods are —

- *Row Major* — Here we slice the matrix between rows, so the first m elements visited belong to the first row, the second m elements to the second row, and so forth. Such an ordering is used inside most modern programming language compilers to represent two-dimensional matrices as a single linear array.

```
(1,1)      row_major(int n, int m)
(1,2)      {
(1,3)              int i,j;        /* counters */
(2,1)
(2,2)              for (i=1; i<=n; i++)
(2,3)                      for (j=1; j<=m; j++)
(3,1)                              process(i,j);
(3,2)      }
(3,3)
```

- *Column Major* — Here we slice the matrix between columns, so the first n elements belong to the first column, the second n elements to the second column, and so forth. This can be done by interchanging the order of the nested loops from row-major ordering. Knowing whether your compiler uses row-major or column-major ordering for matrices is important when optimizing for cache performance and when attempting certain pointer-arithmetic operations.

```
(1,1)       column_major(int n, int m)
(2,1)       {
(3,1)               int i,j;        /* counters */
(1,2)
(2,2)               for (j=1; j<=m; j++)
(3,2)                       for (i=1; i<=n; i++)
(1,3)                               process(i,j);
(2,3)       }
(3,3)
```

- *Snake Order* — Instead of starting each row from the first element, we alternate the order of the directions we travel down the rows. The effect is that of a typewriter which can type both left-to-right and right-to-left so as to minimize printing time.

```
(1,1)       snake_order(int n, int m)
(1,2)       {
(1,3)               int i,j;        /* counters */
(2,3)
(2,2)               for (i=1; i<=n; i++)
(2,1)                   for (j=1; j<=m; j++)
(3,1)                        process(i, j + (m+1-2*j) * ((i+1) % 2));
(3,2)       }
(3,3)
```

- *Diagonal Order* — Here we march up and down diagonals. Note that an $n \times m$ grid has $m + n - 1$ diagonals, each with a variable number of elements. This is a trickier task than it appears at first glance.

```
(1,1)       diagonal_order(int n, int m)
(2,1)       {
(1,2)               int d,j;        /* diagonal and point counters */
(3,1)               int pcount;     /* points on diagonal */
(2,2)               int height;     /* row of lowest point */
(1,3)
(4,1)               for (d=1; d<=(m+n-1); d++) {
(3,2)                       height = 1 + max(0,d-m);
(2,3)                       pcount = min(d, (n-height+1));
(4,2)                       for (j=0; j<pcount; j++)
(3,3)                               process(min(m,d)-j, height+j);
(4,3)               }
            }
```

12.1.2 Dual Graphs and Representations

Two-dimensional arrays are the natural choice to represent planar rectilinear grids. We can let `m[i][j]` denote either the (i, j)th vertex or the (i, j)th face, depending on which

we are interested in. The four neighbors of any cell follow by adding ± 1 to either of the coordinates.

A useful concept in thinking about problems on planar subdivisions is that of the *dual graph*, which has one vertex for each region in the subdivision, and edges between the vertices of any two regions which are neighbors of each other.

The four-color theorem states that every planar map can be colored using at most four colors such that all pairs of neighboring regions are colored differently. However, it *really* is a statement about the number of vertex colors needed to color the dual graph of the map. In fact, the dual graph of any planar subdivision must in itself be a planar graph. Can you show why?

Observe that the dual graphs of both rectangular and hexagonal lattices are slightly smaller rectangular and hexagonal lattices. This is why whatever structure we use to represent vertex connectivities can also be used to represent face connectivities.

An adjacency representation is the natural way to represent an edge-weighted rectilinear grid. This might be most easily done by creating a three-dimensional array `m[i][j][d]`, where d ranges over four values (north, east, south, and west) which denote the edge directions from point (i, j).

12.2 Triangular and Hexagonal Grids

The other regular grids of importance are the *triangular* and *hexagonal* lattices. These are very closely related – indeed, hexagonal grids are essentially triangular grids with alternate vertices removed.

12.2.1 Triangular Lattices

Triangular lattices are constructed from three sets of equally spaced lines, consisting of a horizontal "row" axis, a "column" axis 60° from horizontal, and a "diagonal" axis 120° from horizontal. See Figure 12.1 for an example. Vertices of this lattice are formed by the intersection of three axis lines, so each face of the lattice is an equilateral triangle. Each vertex v is connected to six others, those immediately above and below v on each of the three axes.

To manipulate triangular lattices, we need to be able to identify the proper neighbors of each of these vertices as well as their geographical positions. Doing this requires keeping track of two types of coordinate systems:

- *Triangular/Hexagonal Coordinates* — Here, one vertex is designated as the origin of the grid, point $(0, 0)$. We must assign the coordinates such that the logical neighbors of each vertex are easily obtainable. In a standard rectilinear coordinate system, the four neighbors of (x, y) follow by adding ± 1 to either the row or column coordinates.

  Although the intersection of three lines defines each grid vertex, in fact two dimensions suffice to specify location. We will use the row and column dimensions to define our coordinate system. See Figure 12.1 for an example. A vertex (x, y)

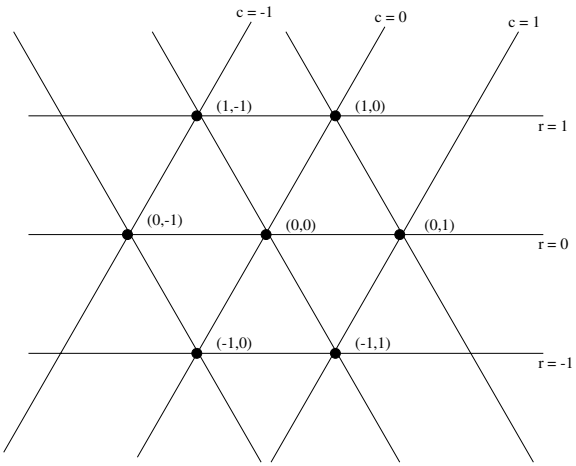


Figure 12.1. A row-column coordinate-system for triangular grids.

lies x rows above the origin, and y (60°)-columns to the right of the origin. The neighbors of a vertex v can be found by adding the following pairs to the coordinates of v, in counterclockwise order: $(0, 1)$, $(1, 0)$, $(1, -1)$, $(0, -1)$, $(-1, 0)$, and $(-1, 1)$.

Similar coordinates can be used to represent the faces of the triangular grid, since the dual graph of the faces will also be a triangular grid. Other coordinate choices are also possible; see [LR76] for an alternative.

- *Geometrical Coordinates* — The vertices of a regular triangular grid also correspond to geometric points in the plane. Note that the grid vertices occur in half-staggered rows in Figure 12.1, because the axis lines are 60° instead of 90° as in rectilinear grids.

  Assume that each lattice point is a distance d from its six nearest neighbors, and that point $(0, 0)$ in triangular coordinates in fact lies at geometric point $(0, 0)$. Then triangular-coordinate point (x_t, y_t) must lie at geometric point

$$(x_g, y_g) = (d(x_t + (y_t \cos(60^o))), dy_t \sin(60^o))$$

  by simple trigonometry. Since $\cos(60^o) = 1/2$ and $\sin(60^o) = \sqrt{3}/2$, we don't even need to use trigonometric functions for this computation.

We will give code for similar manipulations with hexagonal lattices in the next section.

12.2.2 Hexagonal Lattices

Deleting every other vertex from a triangular lattice leaves us with a *hexagonal* lattice, shown in Figure 12.2. Now the faces of the lattice are regular hexagons, and each

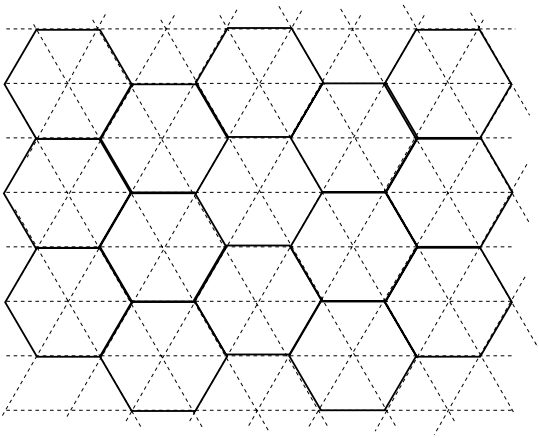


Figure 12.2. Deleting alternate vertices from a triangular lattice yields a hexagonal lattice.

hexagon is adjacent to six other hexagons. The vertices of the lattice now have degree 3, because this lattice is the dual graph of the triangular lattice.

Hexagonal lattices have many interesting and useful properties, primarily because hexagons are "rounder" than squares. Circles enclose the largest amount of area per unit perimeter, so they are the most efficient structures to build. Hexagonal lattices are also more rigid than rectilinear grids, another reason why bees make their honeycombs out of hexagons. Minimizing boundary-per-unit area is also important in minimizing visual artifacts in graphics, so many computer games make images from hexagonal tiles rather than square tiles.

In the previous section, we discussed conversion between triangular/hexagonal coordinates and geometrical coordinates. We assume that the origin of both systems is the center of a disk at $(0, 0)$. The hexagonal coordinate `(xh,yh)` refers to the center of the disk on the horizontal row `xh` and diagonal column `yh`. The geometric coordinate of such a point is a function of the radius of the disk r, half that of the diameter d described in the previous section:

```
hex_to_geo(int xh, int yh, double r, double *xg, double *yg)
{
        *yg = (2.0 * r) * xh * (sqrt(3)/2.0);
        *xg = (2.0 * r) * xh * (1.0/2.0) + (2.0 * r) * yh;
}

geo_to_hex(double xg, double yg, double r, double *xh, double *yh)
{
        *xh = (2.0/sqrt(3)) * yg / (2.0 * r);
        *yh = (xg - (2.0 * r) * (*xh) * (1.0/2.0) ) / (2.0 * r);
}
```

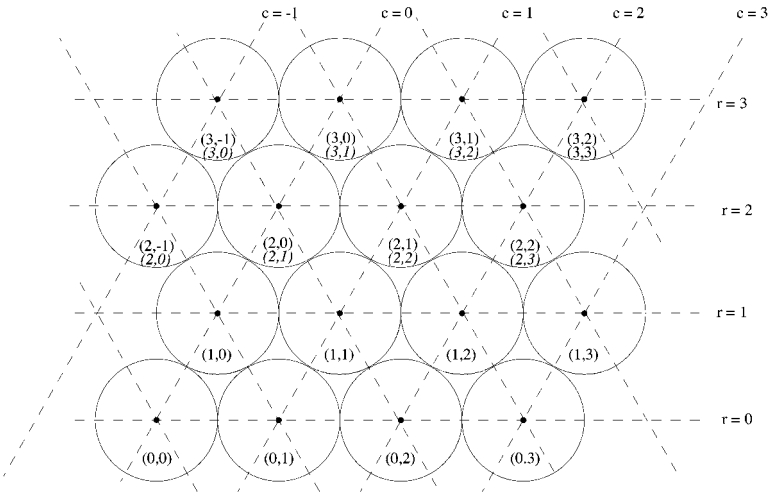


Figure 12.3. A disk packing with hexagonal coordinates, as well as differing array coordinates (below in italics).

The row-column nature of the hexagonal coordinate system implies a very useful property, namely that we can efficiently store a patch of hexagons in a matrix `m[row][column]`. By using the index offsets described for triangular grids, we can easily find the six neighbors of each hexagon.

There is a problem, however. Under the hexagonal coordinate system, the set of hexagons defined by coordinates (hx, hy), where $0 \leq hx \leq x_{\max}$ and $0 \leq hy \leq y_{\max}$, forms a diamond-shaped patch, not a conventional axis-oriented rectangle. However, for many applications we are interested in rectangles instead of diamonds. To solve this problem, we define array coordinates so that `(ax,ay)` refers to the position in an axis-oriented rectangle with $(0, 0)$ as the lower-left-hand point in the matrix:

```
array_to_hex(int xa, int ya, int *xh, int *yh)
{
        *xh = xa;
        *yh = ya - xa + ceil(xa/2.0);
}

hex_to_array(int xh, int yh, int *xa, int *ya)
{
        *xa = xh;
        *ya = yh + xh - ceil(xh/2.0);
}
```

Figure 12.3 shows a hexagonal lattice where each vertex is labeled with both its hexagonal coordinates and, below in italics, its array coordinates if they differ.